**МИНОБРНАУКИ РОССИИ**
**САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ**
**ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ**
**«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)**
**Кафедра МО ЭВМ**

# ОТЧЕТ

**по лабораторной работе №3**

**по дисциплине «Вычислительная математика»**

**Тема: Решение нелинейных уравнений. Метод бисекции**

Студентка гр. 4342 Волкова В.Н.

Преподаватель Пуеров Г.Ю.

Санкт-Петербург

2026

**Цель работы.**

Используя программы - функции BISECT и Round из файла methods.cpp, найти корень уравнения f(x) = 0 методом бисекции с заданной точностью Eps, исследовать зависимость числа итераций от точности Eps при изменении Eps от 0.1 до 0.000001, исследовать обусловленность метода (чувствительность к ошибкам в исходных данных).

**Основные теоретические положения.**

Если найден отрезок [a,b], такой, что $f(a)f(b) < 0$, существует точка c, в которой значение функции равно нулю, т.е. $f(c)=0$, $c \in (a,b)$. Метод бисекции состоит в построении последовательности вложенных друг в друга отрезков, на концах которых функция имеет разные знаки. Каждый последующий отрезок получается делением пополам предыдущего. Процесс построения последовательности отрезков позволяет найти нуль функции f(x) (корень уравнения f(x) = 0 с любой заданной точностью.

Рассмотрим один шаг итерационного процесса. Пусть на (n-1)-м шаге найден отрезок $[a_{n-1}, b_{n-1}] \subset [a, b]$, такой, что $f(a_{n-1})f(b_{n-1}) < 0$. Разделим его пополам точкой $\xi = (a_{n-1} + b_{n-1})/2$ и вычислим $f(\xi)$. Если $f(\xi)=0$, то $\xi = (a_{n-1}+b_{n-1})/2$- корень уравнения. Если $f(\xi) \neq 0$, то из двух половин отрезка выбирается та, на концах которой функция имеет противоположные знаки, поскольку искомый корень лежит на этой половине, т.е.

$a_n=a_{n-1}$, $b_n= \xi$, если $f(\xi)f(a_{n-1}) < 0$ ;

$a_n=\xi$, $b_n= b_{n-1}$ , если $f(\xi)f(a_{n-1}) > 0$ .

Если требуется найти корень с точностью $\varepsilon$, то деление пополам продолжается до тех пор, пока длина отрезка не станет меньше $2\varepsilon$. Тогда координата середины отрезка есть значение корня с требуемой точностью $\varepsilon$.

Метод бисекции является простым и надежным методом поиска простого корня уравнения f(x) = 0 (простым называется корень x=c дифференцируемой функции f(x) , если f(c) и $f'(c) \neq 0$). Этот метод сходится для любых непрерывных функций f(x), в том числе недифференцируемых. Скорость его сходимости

невысока. Для достижения точности $\varepsilon$ необходимо совершить $N \approx \log2(b-a)/\varepsilon$ итераций. Это означает, что для получения каждых трех верных десятичных знаков необходимо совершить около 10 итераций.

**Выполнение работы.**

Вариант 2: функция $f(x) = \arccos(x^2) - x$

1. С помощью графического веб-калькулятора desmos.com построим график (Рисунок 1) для определения отрезка [$a$, $b$] близкого к $f(x) = 0$ на котором выполняется условие $f(a)f(b) < 0$.

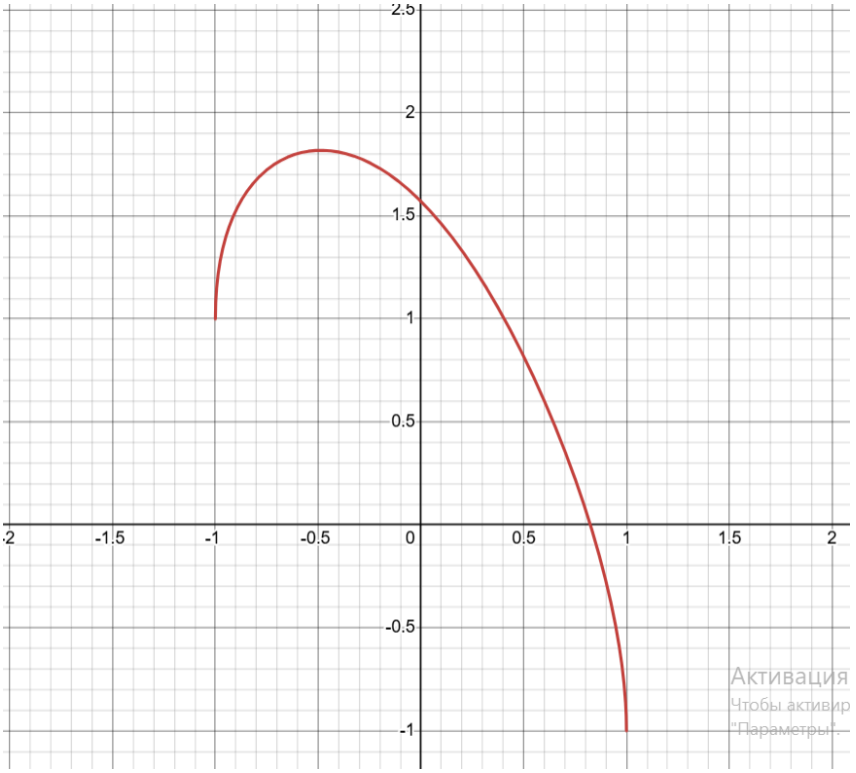


Рисунок 1 — график функции f(x)

На Рисунке 1 видно, что можно взять любой отрезок, где $a \in [-1, 0.82413)$, $b \in (0.82413, 1]$. Возьмем $a = 0$, $b = 1$.

2. В файле lb3.cpp была переопределена функция extern double F(double). Теперь она возвращает acos(x*x) - x , если точность global_delta не задана.

Если задана, то перед возвратом значение округляется функцией Round. Исходный код в Приложении А.

3. Создана функция int main(), где запускаются все вычисления. Они поделены на 2 части: исследование зависимости количества итераций N от точности вычислений Eps и исследование чувствительность метода к ошибкам в исходных данных.
4. Проведены вычисления, построен график зависимости количества итераций N от точности вычислений Eps.

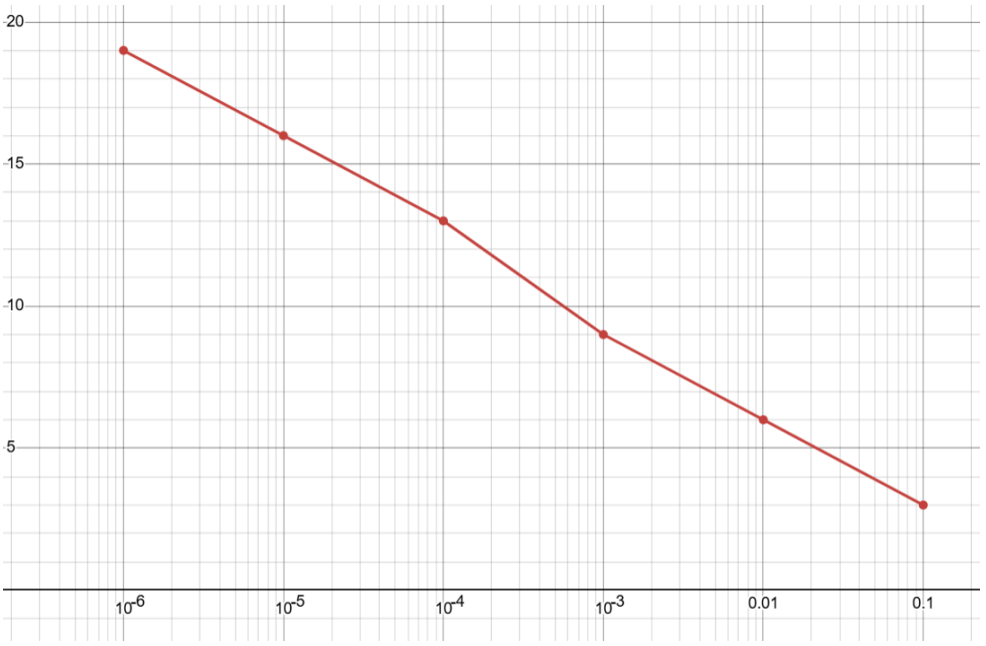


Рисунок 2 — график зависимости N от Eps

По графику видно, что чем более высокая точность вычислений Eps, тем больше итераций N необходимо для нахождения решения уравнения. При этом N растет логарифмически, что подтверждает теоретическую зависимость $N \approx \log2(b-a)/\varepsilon$.

5. Чтобы исследовать чувствительность метода к ошибкам в исходных данных, были запущены вычисления с разными Delta и Eps. Ниже в Таблице 1 приведены результаты вычислений.

Таблица 1 — исследование чувствительности метода к ошибкам в исходных данных

| **global_delta** | $\varepsilon$ | **значение, возвращаемое BISECT** | **N** |
|---|---|---|---|
| $10^{-1}$ | $10^{-1}$ | 0.875 | 3 |
| | $10^{-2}$ | 0.8125 | 3 |
| | $10^{-3}$ | 0.8125 | 3 |
| | $10^{-4}$ | 0.8125 | 3 |
| | $10^{-5}$ | 0.8125 | 3 |
| | $10^{-6}$ | 0.8125 | 3 |
| $10^{-3}$ | $10^{-1}$ | 0.875 | 3 |
| | $10^{-2}$ | 0.828125 | 6 |
| | $10^{-3}$ | 0.824219 | 7 |
| | $10^{-4}$ | 0.824219 | 7 |
| | $10^{-5}$ | 0.824219 | 7 |
| | $10^{-6}$ | 0.824219 | 7 |
| $10^{-5}$ | $10^{-1}$ | 0.875 | 3 |
| | $10^{-2}$ | 0.828125 | 6 |
| | $10^{-3}$ | 0.822266 | 9 |
| | $10^{-4}$ | 0.824097 | 13 |
| | $10^{-5}$ | 0.824142 | 16 |
| | $10^{-6}$ | 0.824131 | 17 |
| $10^{-7}$ | $10^{-1}$ | 0.875 | 3 |
| | $10^{-2}$ | 0.828125 | 6 |
| | $10^{-3}$ | 0.822266 | 9 |
| | $10^{-4}$ | 0.824097 | 13 |

| | | | |
|---|---|---|---|
| | $10^{-5}$ | 0.824142 | 16 |
| | $10^{-6}$ | 0.824133 | 19 |

По результатам исследования можно понять, что точность нахождения решения ограничена точностью вычисления функции. При фиксированном значении погрешности данных Delta дальнейшее уменьшение задаваемой точности вычисления Eps перестает уточнять корень.

При грубых входных данных (Delta > Eps) метод бисекции прекращает работу преждевременно. Это происходит потому, что из-за округления значений функции алгоритм перестает фиксировать разницу знаков на концах отрезков.

**Выводы.**

В ходе работы был изучен метод половинного деления (бисекции) на примере функции $f(x) = \arccos(x^2) - x$, решение которой был успешно найден на отрезке [0, 1] с точностью от 0.1 до 0.000001. Построенный график подтвердил логарифмическую зависимость числа итераций от точности: каждое уменьшение задаваемой точности требует закономерного роста количества вычислений (в среднем на 3–4 итерации), что полностью соответствует теории. Исследование обусловленности показало, что метод является устойчивым к малым погрешностям, однако при грубых погрешностях входных данных процесс уточнения корня прекращается преждевременно, ограничивая итоговую точность результата точностью вычисления самой функции.

# ПРИЛОЖЕНИЕ А
# ИСХОДНЫЙ КОД ПРОГРАММЫ

## Файл lb3.cpp

```
#include <iostream>
#include <cmath>
#include "methods.h"

using namespace std;


double global_delta = 0.0;

double F(double x) {
    double y = acos(x*x) - x;
    if (global_delta > 0) return Round(y, global_delta);
    return y;
}


int main() {
    double left = 0.0, right = 1.0;
    int N;

    cout << "Зависимость N от Eps" << endl;
    for (double eps = 0.1; eps >= 1e-6; eps /= 10) {
        double root = BISECT(left, right, eps, N);
        cout << "Eps:" << eps << "\tN:" << N << "\tRoot:" << root << endl;
    }

    cout << "Влияние Delta" << endl;
    for (double delta = 0.1; delta >= 1e-7; delta /= 10) {
        global_delta = delta;
        cout << "Delta: " << delta << endl;
        for (double eps = 0.1; eps >= 1e-6; eps /= 10) {
            double root = BISECT(left, right, eps, N);
            cout << "Eps:" << eps << "\tN:" << N << "\tRoot:" << root << endl;
        }
    }


    return 0;
}
```

## Файл methods.cpp

```
#include <stdio.h>
#include <math.h>
#include <stdlib.h>

extern double F(double);

double BISECT(double Left,double Right,double Eps,int &N)
{
  double E = fabs(Eps)*2.0;
  double FLeft = F(Left);
  double FRight = F(Right);
```

```
  double X = (Left+Right)/2.0;
  double Y;

  if (FLeft*FRight>0.0) {puts("Неверное задание интервала\n");exit(1);}
  if (Eps<=0.0) {puts("Неверное задание точности\n");exit(1);}

  N=0;
  if (FLeft==0.0)  return Left;
  if (FRight==0.0) return Right;

  while ((Right-Left)>=E)
  {
      X = 0.5*(Right + Left);     /* вычисление середины отрезка       */
      Y = F(X);
      if (Y == 0.0) return (X);
        if (Y*FLeft < 0.0)
           Right=X;
        else
         { Left=X; FLeft=Y; }
        N++;
  };
  return(X);
}

double Round (double X,double Delta)
{
 if (Delta<=1E-9) {puts("Неверное задание точности округления\n");exit(1);}
 if (X>0.0) return (Delta*(long((X/Delta)+0.5)));
    else    return (Delta*(long((X/Delta)-0.5)));
}

double ITER(double X0,double Eps,int &N)
{
  if (Eps<=0.0) {puts("Неверное задание точности\n");exit (1);}
  double X1=F(X0);
  double X2=F(X1);
  N = 2;
  while( (X1 - X2)*(X1 - X2) > fabs((2*X1-X0-X2)*Eps) )
  {
      X0 = X1;
      X1 = X2;
      X2 = F(X1);
        N++;
  }
   return(X2);
}

#ifdef __NEWTON
double NEWTON (double X,double Eps,int &N)
{
  extern double F1 (double);
  double Y,Y1,DX;
  N=0;
  do
  {
    Y  = F(X);
    if (Y==0.0) return (X);

    Y1 = F1(X);
    if (Y1==0.0) {puts("Производная обратилась в ноль\n");exit(1);}

    DX=Y/Y1; X=X-DX; N++;
  }
```

```
  while (fabs(DX)>Eps);
  return (X);
}
#endif

double HORDA(double Left,double Right,double Eps,int &N)
{
  double FLeft = F(Left);
  double FRight = F(Right);
  double X,Y;

  if (FLeft*FRight>0.0) {puts("Неверное задание интервала\n");exit(1);}
  if (Eps<=0.0) {puts("Неверное задание точности\n");exit(1);}

  N=0;
  if (FLeft==0.0)  return Left;
  if (FRight==0.0) return Right;

  do
  {
      X = Left-(Right-Left)*FLeft/(FRight-FLeft);
      Y = F(X);
      if (Y == 0.0) return (X);
        if (Y*FLeft < 0.0)
         { Right=X; FRight=Y; }
        else
         { Left=X; FLeft=Y; }
        N++;
  }
  while ( fabs(Y) >= Eps );

  return(X);

}
```

## Файл methods.h

```
extern double F(double);
/*****************************************************************/
/*           Функция F (X) , задаваемая пользователем            */
/*****************************************************************/

#ifdef __NEWTON
extern double F1(double);
/*****************************************************************/
/*     Производная функции F (X) , задаваемая пользователем      */
/*****************************************************************/
#endif

double Round (double X,double Delta);
/*****************************************************************/
/*   Функция Round (X,Delta) , предназначена для округления      */
/*                    X с точностью Delta                        */
/*****************************************************************/

double BISECT(double Left,double Right,double Eps,int &N);
/*****************************************************************/
/*    Функция BISECT предназначена для решения уравнения F(X)=0  */
/*    методом деления отрезка пополам. Использованы обозначения: */
/*      Left - левый конец промежутка                            */
/*      Right - правый конец промежутка                          */
/*      Eps - погрешность вычисления корня уравнения;            */
```

```
/*      N - число итераций                                          */
/*****************************************************************/

double ITER(double X0,double Eps,int &N);
/*****************************************************************/
/*    Функция ITER предназначена для решения уравнения F(X)=X     */
/*      методом простой итерации. Использованы обозначения:       */
/*      X0 - начальное приближение корня                          */
/*      Eps - погрешность вычисления корня уравнения;             */
/*      N - число итераций                                        */
/*****************************************************************/

double HORDA(double Left,double Right,double Eps,int &N);
/*****************************************************************/
/*    Функция HORDA предназначена для решения уравнения F(x)=0    */
/*      методом хорд. Использованы обозначения:                   */
/*      Left - левый конец промежутка                             */
/*      Right - правый конец промежутка                           */
/*      Eps - погрешность вычисления корня уравнения;             */
/*      N - число итераций                                        */
/*****************************************************************/

double NEWTON (double X,double Eps,int &N);
/*****************************************************************/
/*  Функция NEWTON предназначена для решения уравнения F(X)=0     */
/*        методом касательных. Использованы обозначения:          */
/*      X - начальное приближение корня                           */
/*      Eps - погрешность вычисления корня уравнения;             */
/*      N - число итераций                                        */
/*****************************************************************/

#include "methods.cpp"
```